BLOQUE 2

Colecciones y funciones

Las estructuras de datos de Python y la forma de organizar el código.

bloque2.py

```python
muestras = [
    {"petalo": 1.4, "clase": "setosa"},
    {"petalo": 4.7, "clase": "versicolor"},
]

clases = {m["clase"] for m in muestras}
```

1

Las cuatro colecciones

Elegir bien la estructura es la mitad del algoritmo. En Python la elección es explícita y barata.

Cuándo usar cada una

Colección	Sintaxis	¿Mutable?	Duplicados	Caso típico en IA
list	[1, 2, 3]	Sí	Sí	Secuencia de muestras o de predicciones
tuple	(1, 2, 3)	No	Sí	Registro fijo: (x, y), forma de una matriz
dict	{"a": 1}	Sí	Claves no	Una fila con nombre de columna; conteos
set	{1, 2, 3}	Sí	No	Etiquetas distintas, vocabulario, deduplicar

Listas: la colección por defecto

Heterogéneas, dinámicas y con índices negativos.

- Índice 0 al principio; -1 es el último elemento.
- Crecen y encogen: no hay que dimensionarlas.
- Pueden mezclar tipos, aunque casi nunca conviene.
- in comprueba pertenencia; len() da el tamaño.

listas.py

```
notas = [7.5, 9.0, 4.2, 6.8]

notas[0]         # 7.5
notas[-1]        # 6.8  el ultimo
len(notas)       # 4

notas.append(8.1)
notas[2] = 5.0   # mutable

9.0 in notas     # True

mixta = [1, "dos", 3.0, True, None]
```

LISTAS

Slicing: rebanadas [inicio:fin:paso]

slicing.py

```
datos = [10, 20, 30, 40, 50, 60]

datos[1:4]      # [20, 30, 40]     el final NO se incluye
datos[:3]       # [10, 20, 30]     desde el principio
datos[3:]       # [40, 50, 60]     hasta el final
datos[-2:]      # [50, 60]         los dos ultimos
datos[::2]      # [10, 30, 50]     uno de cada dos
datos[::-1]     # [60, 50, ...]    invertida

# Uso real: separar entrenamiento y test
corte = int(len(datos) * 0.8)
entrenamiento, prueba = datos[:corte], datos[corte:]
```

Clave: el slicing devuelve una lista nueva; también funciona sobre cadenas y tuplas. datos[:] es la forma corta de copiar una lista.

Métodos útiles

metodos_lista.py

```
xs = [3, 1, 2]

xs.append(4)         # [3, 1, 2, 4]      añade al final
xs.extend([5, 6])    # [3, 1, 2, 4, 5, 6] concatena otra lista
xs.insert(0, 0)      # inserta en la posición indicada
xs.remove(3)         # borra la PRIMERA aparición del valor 3
xs.pop()             # saca y devuelve el último
xs.index(2)          # posición del valor 2
xs.count(2)          # cuántas veces aparece

xs.sort()                        # ordena EN EL SITIO, devuelve None
ordenada = sorted(xs, reverse=True)    # devuelve una lista nueva

xs.sort(key=len)     # ordenar por criterio: longitud, campo, etc.
```

Error frecuente: xs = xs.sort() deja xs valiendo None. sort() modifica; sorted() devuelve. Esa distinción recorre toda la biblioteca estándar.

MUTABILIDAD

Copiar una lista no es asignarla

Alias: la misma lista con dos nombres

alias.py

```
a = [1, 2, 3]
b = a            # NO copia

b.append(4)

print(a)         # [1, 2, 3, 4]
print(a is b)    # True

# Efecto colateral silencioso:
def limpiar(datos):
    datos.clear()   # modifica el original
```

Copia real

copia.py

```
a = [1, 2, 3]
b = a.copy()     # o list(a) o a[:]

b.append(4)

print(a)         # [1, 2, 3]
print(a is b)    # False

# Listas anidadas: copia profunda
from copy import deepcopy
c = deepcopy(matriz)
```

Idea clave: los objetos mutables se pasan a las funciones por referencia. Si una función no debe modificar su entrada, copia dentro o devuelve una estructura nueva.

Tuplas y desempaquetado

Inmutables: sirven como registro fijo y como clave de diccionario.

- Misma indexación y slicing que las listas.
- El desempaquetado evita índices ilegibles.
- Una función devuelve varios valores con una tupla.
- Al ser inmutables pueden usarse como clave de un dict.

tuplas.py

```
punto = (3.5, 7.2)
x, y = punto            # desempaquetado

a, b = 1, 2
a, b = b, a             # intercambio sin temporal

primero, *resto = [1, 2, 3, 4]
# primero=1, resto=[2, 3, 4]

def minmax(xs):
    return min(xs), max(xs)

bajo, alto = minmax([4, 9, 1])
```

DICCIONARIOS

Diccionarios: pares clave → valor

La estructura más usada de Python: una fila de datos con nombres.

- Acceso en tiempo constante por clave.
- d[clave] lanza KeyError; d.get(clave, defecto) no.
- Claves inmutables: str, int, tupla. Valores, cualquier cosa.
- Desde 3.7 conservan el orden de inserción.

diccionarios.py

```
flor = {"largo": 5.1, "clase": "setosa"}

flor["largo"]             # 5.1
flor["ancho"] = 3.5       # añade clave

flor.get("color")         # None, sin error
flor.get("color", "n/d") # "n/d"

"clase" in flor           # True
del flor["ancho"]

flor.keys(); flor.values(); flor.items()
```

Recorrer y acumular: el patrón de conteo

conteo.py

```python
registros = [
    {"clase": "setosa", "largo": 5.1},
    {"clase": "virginica", "largo": 6.3},
    {"clase": "setosa", "largo": 4.9},
]

conteo = {}
for r in registros:
    clave = r["clase"]
    conteo[clave] = conteo.get(clave, 0) + 1     # patrón de conteo

print(conteo)            # {'setosa': 2, 'virginica': 1}

for clase, n in conteo.items():                  # recorrer pares
    print(f"{clase:>12}: {n}")

from collections import Counter                  # atajo de la biblioteca
print(Counter(r["clase"] for r in registros))
```

Por qué importa: este patrón (lista de diccionarios + conteo por clave) es literalmente lo que hace un preprocesado de datos antes de entrenar un modelo.

Conjuntos: sin duplicados y con álgebra

Pertenencia rápida y operaciones de teoría de conjuntos.

- set(lista) elimina duplicados en una línea.
- No mantienen orden ni admiten indexación.
- Unión |, intersección &, diferencia -.
- Comprobar pertenencia es mucho más rápido que en una lista.

conjuntos.py

```
etiquetas = ["a", "b", "a", "c"]
unicas = set(etiquetas)      # {'a', 'b', 'c'}

vacio = set()                # ¡no {} : eso es un dict!

real = {"a", "b", "c"}
pred = {"a", "c", "d"}

real & pred                  # {'a', 'c'} aciertos
real - pred                  # {'b'}      no detectadas
real | pred                  # todas

len(real & pred) / len(real) # cobertura
```

PRÁCTICA 2.1

Ejercicios de colecciones

- E2.1 · Estadísticas de una lista de notas: media, máximo, mínimo y cuántas aprobadas, sin usar statistics.
- E2.2 · Deduplicar conservando el orden de aparición original. Es decir, elimina las copias repetidas de una lista.
- E2.3 · Agenda: diccionario de contactos con alta, baja, búsqueda y listado ordenado, en un bucle de menú.
- E2.4 · Frecuencia de palabras de un texto: normaliza, separa y cuenta; muestra el top 5.

e2_4.py

```
SIGNOS = ".,:;!?¡¿()\"'"

def contar_palabras(texto: str) -> dict[str, int]:


    limpio = texto.lower()
    for s in SIGNOS:
        limpio = limpio.replace(s, " ") # quitar puntuación
# antes de separar
    conteo: dict[str, int]={}
    ...
```

2

Iteración pythónica

Mismo resultado, la mitad de líneas y menos sitios donde equivocarse.

COMPRENSIONES

Comprensión de listas: el idioma de Python

Como en cualquier lenguaje

bucle.py

```
cuadrados = []
for n in range(10):
    cuadrados.append(n ** 2)

pares = []
for n in numeros:
    if n % 2 == 0:
        pares.append(n)

nombres = []
for r in registros:
    nombres.append(r["clase"])
```

Como en Python

comprension.py

```
cuadrados = [n ** 2 for n in range(10)]

pares = [n for n in numeros if n % 2 == 0]

nombres = [r["clase"] for r in registros]

# con transformación y filtro a la vez
escaladas = [
    x / maximo
    for x in medidas
    if x is not None
]
```

Idea clave: [expresión for elemento in iterable if condición]. Si no cabe en dos líneas legibles, usa un bucle normal: la legibilidad manda sobre la brevedad.

COMPRENSIONES

También de diccionario, de conjunto y anidadas

comprensiones.py

```python
# Diccionario: nombre -> longitud
longitudes = {nombre: len(nombre) for nombre in nombres}

# Conjunto: clases distintas presentes en el dataset
clases = {r["clase"] for r in registros}

# Filtrar un diccionario
aprobados = {a: n for a, n in notas.items() if n >= 5}

# Anidada: aplanar una matriz
matriz = [[1, 2], [3, 4], [5, 6]]
plano = [x for fila in matriz for x in fila]      # [1, 2, 3, 4, 5, 6]

# Expresión generadora: no construye la lista, ahorra memoria
total = sum(x ** 2 for x in range(1_000_000))
```

Generadores: los paréntesis en vez de corchetes crean un generador: produce los valores de uno en uno. Imprescindible con ficheros o datasets grandes.

zip: recorrer varias listas a la vez

Empareja los elementos por posición y devuelve tuplas.

- Sustituye al for i in range(len(a)) con a[i] y b[i].
- Se detiene en la secuencia más corta, sin avisar.
- strict=True (3.10+) lanza error si las longitudes difieren.
- Admite tres o más secuencias a la vez.

zip.py

```python
nombres = ["ana", "leo", "eva"]
notas   = [7.5, 9.0, 6.2]

for nombre, nota in zip(nombres, notas):
    print(f"{nombre}: {nota}")

# Lo que evita:
for i in range(len(nombres)):
    print(nombres[i], notas[i])

list(zip([1, 2, 3], "ab"))
# [(1, 'a'), (2, 'b')]   ¡el 3 se pierde!

zip(nombres, notas, strict=True)
```

zip en la práctica

zip_practica.py

```
y_real = ["setosa", "virginica", "setosa", "versicolor"]
y_pred = ["setosa", "versicolor", "setosa", "versicolor"]

aciertos = sum(r == p for r, p in zip(y_real, y_pred))    # 3
print(f"Acierto: {aciertos / len(y_real):.0%}")            # 75%

columnas = ["largo", "ancho", "clase"]
fila = [5.1, 3.5, "setosa"]
dict(zip(columnas, fila))      # {'largo': 5.1, 'ancho': 3.5, 'clase': 'setosa'}

pares = [("ana", 7), ("leo", 9)]
nombres, notas = zip(*pares)   # "descomprimir": ('ana', 'leo') y (7, 9)

temps = [18, 21, 19, 24]
[b - a for a, b in zip(temps, temps[1:])]    # [3, -2, 5]  diferencias consecutivas

for i, (x, y) in enumerate(zip(xs, ys)):     # índice y dos listas a la vez
    ...
```

La primera suma es exactamente el cálculo del acierto de un clasificador, y dict(zip(...)) es cómo una fila de CSV se convierte en diccionario.

HERRAMIENTAS

Funciones integradas que ahorran bucles

integradas.py

```python
for i, fila in enumerate(datos, start=1):    # índice + valor
    print(i, fila)

for nombre, nota in zip(nombres, notas):     # recorrer en paralelo
    print(nombre, nota)

sum(notas); max(notas); min(notas); len(notas)
sum(notas) / len(notas)                      # media

any(n < 0 for n in notas)                    # ¿hay alguno negativo?
all(0 <= n <= 10 for n in notas)             # ¿son todos válidos?

sorted(registros, key=lambda r: r["largo"], reverse=True)[:3]
max(conteo, key=conteo.get)                  # clase más frecuente
```

El parámetro key: key recibe una función que extrae el criterio de orden. Es el mismo mecanismo que usarás para ordenar predicciones por probabilidad.

PRÁCTICA 2.2 ·

Reescribe en Python idiomático

- E2.5 · Normaliza una lista de valores al rango 0-1 con una sola comprensión.
- E2.6 · Dado un texto, construye {palabra: frecuencia} y devuelve las 3 más repetidas usando sorted con key.
- E2.7 · Con zip: calcula el acierto entre etiquetas reales y predichas, y crea un dict a partir de columnas y valores.

e2_5.py

```
# E2.5  normalización min-max
valores = [12, 7, 30, 18]

lo, hi = min(valores), max(valores)

norm = [... for v in valores]
# [0.17, 0.0, 1.0, 0.48]
```

```
# Ejemplos de etiquetas reales y predichas para el E2.7

Y_REAL=[“setosa”,”virginica”,”setosa”,”versicolor”]

Y_REAL=[“setosa”,”virginica”,”setosa”,”setosa”]
```

3

Funciones

Parámetros flexibles, documentación integrada y funciones tratadas como cualquier otro valor.

DEFINICIÓN

def, return y varios valores de vuelta

Sin tipos obligatorios en la firma y sin declaración de excepciones.

- Si no hay return, la función devuelve None.
- Devolver varios valores es devolver una tupla.
- Los type hints documentan la intención sin forzarla.
- El docstring va justo debajo del def, entre triples comillas.

funciones.py

```python
def media(valores: list[float]) -> float:
    """Media aritmética de una lista."""
    return sum(valores) / len(valores)


def resumen(valores):
    return min(valores), media(valores), max(valores)


lo, m, hi = resumen([4, 8, 6])

help(media)    # muestra el docstring
```

PARÁMETROS

Posicionales, nombrados y por defecto

parametros.py

```python
def entrenar(datos, epocas=10, tasa=0.01, verbose=False):
    ...

entrenar(X)                          # usa todos los valores por defecto
entrenar(X, 50)                       # posicional: epocas = 50
entrenar(X, tasa=0.1)                 # nombrado: se salta epocas
entrenar(X, verbose=True, epocas=5)   # los nombrados van en cualquier orden

# Los parámetros con valor por defecto van SIEMPRE los últimos
# def malo(a=1, b): ...   -> SyntaxError

config = {"epocas": 20, "tasa": 0.05}
entrenar(X, **config)                 # desempaqueta el dict como argumentos
```

Buena práctica: los argumentos nombrados son documentación viva: entrenar(X, 50, 0.1, True) no se entiende; entrenar(X, epocas=50, verbose=True) sí.

TRAMPA CLÁSICA

Nunca uses una lista como valor por defecto

Mal: el valor por defecto se comparte

trampa.py

```
def anotar(x, registro=[]):
    registro.append(x)
    return registro

anotar(1)     # [1]
anotar(2)     # [1, 2]  ¡sorpresa!
anotar(3)     # [1, 2, 3]

# El [] se crea UNA vez, al definir
# la función, no en cada llamada.
```

Bien: centinela None

correcto.py

```
def anotar(x, registro=None):
    if registro is None:
        registro = []
    registro.append(x)
    return registro

anotar(1)     # [1]
anotar(2)     # [2]
anotar(3)     # [3]

# Regla: por defecto solo inmutables
# (None, números, cadenas, tuplas)
```

Para recordar: este error aparece en entrevistas técnicas y en bugs reales de producción. Explica por qué: el cuerpo de la función se evalúa en cada llamada, la firma no.

FLEXIBILIDAD

*args y **kwargs: número variable de argumentos

variables.py

```python
def total(*numeros):              # args -> tupla
    return sum(numeros)

total(1, 2, 3)                    # 6
total(*[1, 2, 3, 4])              # desempaqueta la lista


def registrar(evento, **datos):  # kwargs -> diccionario
    print(evento, datos)

registrar("fin_epoca", perdida=0.21, acierto=0.93)
# fin_epoca {'perdida': 0.21, 'acierto': 0.93}


def envoltorio(*args, **kwargs):     # firma universal
    return funcion_real(*args, **kwargs)
```

Ojo: el asterisco no es el nombre: * empaqueta posicionales en una tupla y ** empaqueta nombrados en un diccionario. Verás esta firma en todos los decoradores.

ÁMBITO

Dentro y fuera de la función

Lo asignado dentro de una función es local salvo declaración expresa.

- Se puede leer una variable global, pero no reasignarla.
- global funciona, pero casi siempre indica un mal diseño.
- Mejor: recibir por parámetro y devolver con return.
- Regla LEGB: Local, Enclosing, Global, Builtins.

ambito.py

```
contador = 0

def sumar():
    contador = contador + 1    # UnboundLocalError

def sumar_ok(c):
    return c + 1               # sin efectos colaterales

contador = sumar_ok(contador)

UMBRAL = 0.5                   # constante: leer, no tocar

def clasificar(p):
    return "si" if p > UMBRAL else "no"
```

FUNCIONES COMO DATOS

lambda: funciones anónimas de una línea

lambda parámetros: expresión. Sin nombre, sin return y sin bloque.

- Devuelve automáticamente el valor de la expresión.
- Admite varios parámetros y valores por defecto.
- Solo una expresión: ni bucles, ni asignaciones, ni varias líneas.
- Si hay que nombrarla o reutilizarla, usa def.

lambda.py

```
def cuadrado(x):
    return x ** 2

cuadrado = lambda x: x ** 2   # equivalente, desaconsejado

(lambda a, b: a + b)(2, 3)    # 5
(lambda x, n=2: x ** n)(3)    # 9

paridad = lambda n: "par" if n % 2 == 0 else "impar"

OPS = {"suma":  lambda a, b: a + b,
       "resta": lambda a, b: a - b}
OPS["resta"](7, 4)            # 3
```

Donde brilla lambda: como argumento

lambda_key.py

```python
alumnos = [{"nombre": "ana", "nota": 7.5, "edad": 22},
           {"nombre": "leo", "nota": 9.0, "edad": 19},
           {"nombre": "eva", "nota": 7.5, "edad": 20}]

sorted(alumnos, key=lambda a: a["nota"], reverse=True)     # nota descendente
sorted(alumnos, key=lambda a: (-a["nota"], a["nombre"]))  # nota desc., nombre asc.
max(alumnos, key=lambda a: a["nota"])["nombre"]            # 'leo'
min(alumnos, key=lambda a: a["edad"])                       # la persona más joven

palabras = ["modelo", "IA", "red", "neurona"]
sorted(palabras, key=len)                          # una función existente también vale
sorted(palabras, key=lambda p: p.lower())          # sin distinguir mayúsculas

puntos = [(3, 4), (1, 1), (0, 5)]
sorted(puntos, key=lambda p: (p[0] ** 2 + p[1] ** 2) ** 0.5)    # distancia al origen
```

key=lambda es el uso número uno de lambda.

MAP

map: aplicar una función a cada elemento

map(función, iterable) devuelve un iterador con la función aplicada a cada elemento.

map.py

```
precios = [10.0, 24.5, 8.99]

con_iva = map(lambda p: round(p * 1.21, 2), precios)
print(con_iva)          # <map object at 0x...>   es perezoso, como range
print(list(con_iva))    # [12.1, 29.64, 10.88]

list(map(float, ["3.5", "2", "7.1"]))       # [3.5, 2.0, 7.1]   función existente
list(map(str.upper, ["ana", "leo"]))        # ['ANA', 'LEO']    método como función

a, b = [1, 2, 3], [10, 20, 30]
list(map(lambda x, y: x + y, a, b))         # [11, 22, 33]   varios iterables

[round(p * 1.21, 2) for p in precios]      # equivalente con comprensión
```

map devuelve un iterador que se consume una sola vez: si haces list() dos veces sobre el mismo objeto, la segunda lista sale vacía.

FILTER

filter: quedarse con los que cumplen

filter(función, iterable) conserva los elementos para los que la función devuelve algo verdadero.

filter.py

```
notas = [7.5, 4.0, 9.2, 5.5, 3.9, 8.1]
aprobadas = list(filter(lambda n: n >= 5, notas))    # [7.5, 9.2, 5.5, 8.1]

palabras = ["dato", "", "modelo", None, "red", ""]
list(filter(None, palabras))     # ['dato', 'modelo', 'red']   quita los "falsos"

registros = [{"clase": "setosa", "largo": 5.1},
             {"clase": "virginica", "largo": None},
             {"clase": "setosa", "largo": 4.9}]
validos = list(filter(lambda r: r["largo"] is not None, registros))

# map y filter encadenados: cuadrados de los pares
list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, range(10))))
# [0, 4, 16, 36, 64]
[x ** 2 for x in range(10) if x % 2 == 0]           # lo mismo, comprensión
```

filter(None, datos) es un atajo habitual para descartar valores vacíos antes de procesar un dataset.

REDUCE

reduce: acumular hasta un único valor

reduce(función, iterable, inicial) combina los elementos de dos en dos arrastrando el resultado.

reduce.py

```python
from functools import reduce          # desde Python 3 no es integrada

reduce(lambda acc, x: acc + x, [1, 2, 3, 4])         # 10
# paso a paso: ((1 + 2) + 3) + 4

reduce(lambda acc, x: acc * x, [1, 2, 3, 4], 1)      # 24   producto, empieza en 1
reduce(lambda a, b: a if a > b else b, [4, 9, 2])    # 9    máximo manual

palabras = ["python", "para", "ia"]
reduce(lambda acc, p: acc + len(p), palabras, 0)     # 12   acc empieza en 0

conteos = [{"a": 1}, {"b": 2}, {"a": 3}]
reduce(lambda t, d: {**t, **d}, conteos, {})         # {'a': 3, 'b': 2}  fusiona

# Casi siempre hay algo más claro: sum(), math.prod(), max(), "".join()
```

Cuidado: sin valor inicial, reduce lanza TypeError con una lista vacía. Pon siempre el inicial si la lista puede llegar vacía.

ESTILOS

El mismo cálculo, dos estilos

Funcional: filter + map + reduce

funcional.py

```python
from functools import reduce

datos = [3, -1, 4, -5, 2]

positivos = filter(lambda x: x > 0, datos)
cuadrados = map(lambda x: x ** 2, positivos)
total = reduce(lambda a, b: a + b,
               cuadrados, 0)

print(total)      # 29
```

Pythónico: comprensión + sum

pythonico.py

```python
datos = [3, -1, 4, -5, 2]

total = sum(x ** 2 for x in datos if x > 0)

print(total)      # 29


# map con funciones existentes
# también se lee bien:
nums = list(map(float, texto.split(",")))
```

Criterio: en Python se prefieren comprensiones y sum/max/min. map y filter compensan con funciones ya definidas; reduce, cuando no hay una integrada. En Spark verás el estilo funcional a diario.

PRÁCTICA 2.3

Ejercicios: lambda, map, filter y reduce

- E2.8 · Ordena productos (nombre, precio, stock) por precio y, a igual precio, por nombre.
- E2.9 · Con map: convierte ["3,5", "2,25", "7"] en floats cambiando la coma por punto.
- E2.10 · Con filter: quédate con los correos que tienen "@" y un punto después de ella.
- E2.11 · Con reduce: el producto de una lista y la palabra más larga de un texto.
- E2.12 · Encadena filter, map y reduce: total con IVA de los precios mayores de 10 €.
- E2.13 · Reescribe E2.16 con una comprensión y sum. ¿Cuál se lee mejor?

e2_12.py

```python
# E2.12  esqueleto
from functools import reduce

precios = [4.5, 12.0, 25.9, 8.0, 15.5]

caros   = filter(lambda p: ..., precios)
con_iva = map(lambda p: ..., caros)
total   = reduce(lambda acc, p: ...,
                 con_iva, 0)

print(f"{total:.2f} €")     # 64.61 €
```

4

Proyecto 1 · Analizador de datos

Construir, sin librerías externas, el preprocesado que luego hace pandas en una línea.

PROYECTO 1 · ENUNCIADO

Analizador de un conjunto de flores

- Partes de un dataset embebido en el código como texto (formato CSV, 30 filas del conjunto Iris).
- 1 · Parsear el texto a una lista de diccionarios con tipos correctos.
- 2 · Calcular, por clase: número de muestras, media, mínimo y máximo de cada medida.
- 3 · Normalizar las medidas al rango 0-1 con una comprensión.
- 4 · Detectar y listar los valores atípicos (a más de 2 desviaciones de la media).
- 5 · Imprimir un informe alineado con f-strings.

proyecto1.py

```
DATOS = """5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
7.0,3.2,4.7,1.4,versicolor"""

COLUMNAS = ["largo_sep", "ancho_sep",
            "largo_pet", "ancho_pet",
            "clase"]

def parsear(texto):
    """Devuelve lista de dicts."""
    ...
```